

Jenkins + Gitee实现CI/CD

Jenkins + Gitee实现CI/CD

1. 前提准备

- Linux服务器
- 服务器安装好Docker

2. 安装Jenkins

2.1 拉取镜像

[jenkins/jenkins Tags](#) | [Docker Hub](#)

版本越高越好，不然有的Jenkins插件用不了

```
docker pull jenkins/jenkins:2.505
```

下载不下来怎么办？

- 镜像加速，以阿里云为例：[配置镜像加速器_容器镜像服务\(ACR\)-阿里云帮助中心](#)
- 还不行，本地开代理下载，上传到服务器

```
# 本地下载
docker pull jenkins/jenkins:2.505
docker save jenkins/jenkins:2.505 > jenkins-2.505.tar
# 服务器
docker load < jenkins-2.505.tar
docker images | grep jenkins/jenkins
```

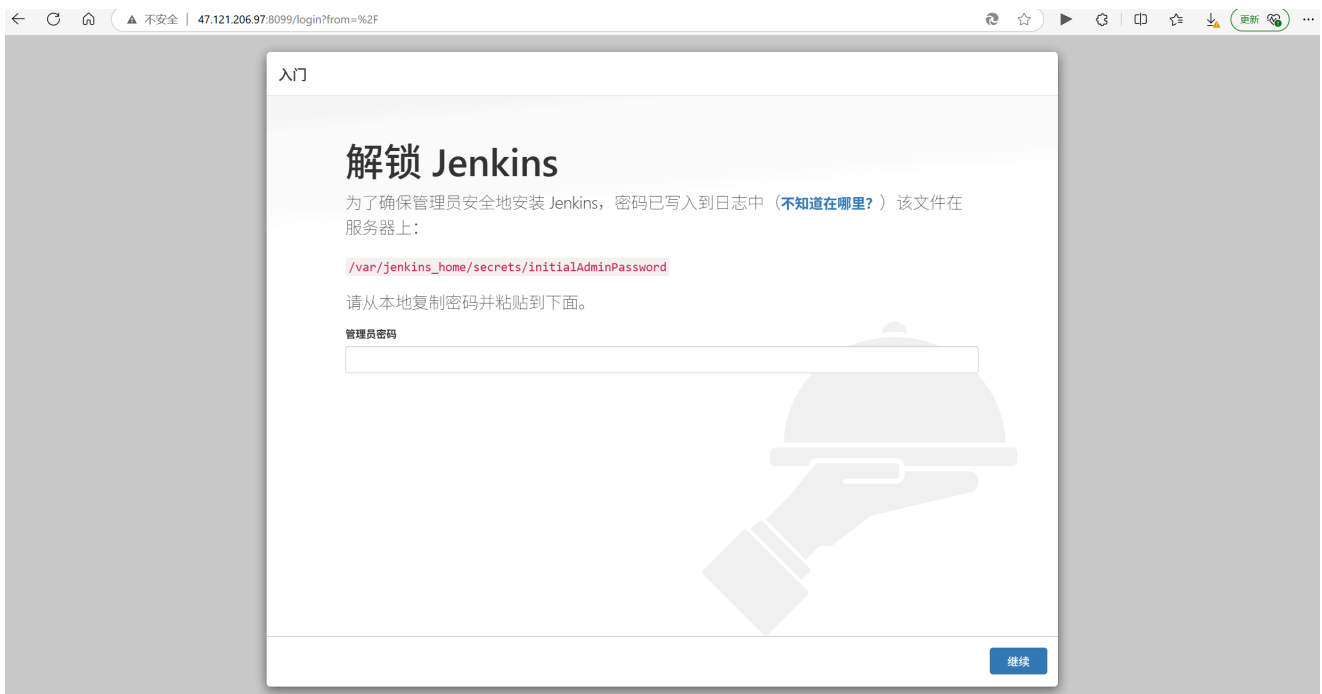
2.2 运行容器

```
# 挂载目录
mkdir -p /usr/local/jenkins
chmod 777 /usr/local/jenkins
# 运行容器 映射到8099端口了 记得修改防火墙和服务器安全组 容器名字是 myjenkins
docker run -d -p 8099:8080 -p 50099:50000 \
  -v /usr/local/jenkins:/var/jenkins_home \
  --name myjenkins \
```

```
jenkins/jenkins:2.505
# 查看是否运行成功
docker ps
# 查看日志
docker logs -f myjenkins
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a45995a21024	jenkinsci/blueocean	"/sbin/tini -- /usr/..."	7 seconds ago	Up 6 seconds	0.0.0.0:8099->8080/tcp, [::]:8099->8080/tcp, 0.0.0.0:50099->50000/tcp, [::]:50099->50000/tcp	myjenkins

出现上面这条是运行起来了，现在打开ip:8099就会显示以下页面



这个时候可以不着急下一步操作，先配置镜像加速，不然下载插件太慢

```
vim /usr/local/jenkins/hudson.model.UpdateCenter.xml
```

修改为：

```
<?xml version='1.1' encoding='UTF-8'?>
<sites>
  <site>
    <id>default</id>
    <url>https://mirrors.huaweicloud.com/jenkins/updates/update-
center.json</url>
  </site>
</sites>
```

然后重启服务

```
docker stop myjenkins
docker start myjenkins
```

2.3 其他配置

上面的页面让查看 `/var/jenkins_home/secrets/initialAdminPassword`，实际上容器的 `/var/jenkins_home` 对应服务器的 `/usr/local/jenkins`，所以需要查看 `/usr/local/jenkins/secrets/initialAdminPassword`

```
cat /usr/local/jenkins/secrets/initialAdminPassword
```

登录进去要等待一会...下载插件还需要一段时间...

创建一个root账户：

新手入门

创建第一个管理员用户

Username:

Password:

Confirm password:

Full name:

E-mail address:

Jenkins 2.346.3 [使用admin账户继续](#) [保存并完成](#)

注意：页面显示重启jenkins后，如果直接无法访问了，需要检查容器是不是停了，如果停了，在服务器启动容器。

```
docker start myjenkins
```

如果页面出现403：换个网络试试/参考[jenkins报错403的解决方案-腾讯云开发者社区-腾讯云](#)

其他的配置参考链接完成以下：[Jenkins + Gitee 实现代码自动化构建（超级详细）-腾讯云开发者社区-腾讯云](#)

- 配置gitee的令牌
- 配置好仓库webhook

- 编写shell/其他命令（视情况而定）

如果push代码到gitee仓库，也设置好了webhook，还是无法自动触发构建？

使用通用触发插件，参考：[jenkins使用gitee插件自动部署webhook404问题记录_jenkins gitee 插件-CSDN博客](#)

一个项目一个token

2.4 需要注意

1. 我打算把所有的项目都放到/service文件夹下，为了使容器内部的改动可以同步到服务器环境中，还需要另外挂载目录。不像我这样配置也可以，jenkins有默认的 /var/jenkins_home/workspace 目录，你可以使用那个目录对应的容器外目录。

```
# 挂载目录
mkdir -p /service
chmod 777 /service
sudo chown -R 1000:1000 /service

docker run -d -p 8099:8080 -p 50099:50000 \
  -v /usr/local/jenkins:/var/jenkins_home \
  -v /service:/service \
  --name myjenkins \
  jenkins/jenkins:2.505
```

2. 因为是容器部署的jenkins，那么在执行shell时，会默认在容器内的环境执行。但是预期是在宿主机运行服务，所以需要下载插件 Publish Over SSH，单独进行ssh配置：

Dashboard > 系统管理 > System >

SSH Server

Name ?

给这个 SSH 起一个名字 ①

① Required. Cannot contain < & ' " \

Hostname ?

服务器的地址 就是公网IP ②

① 必填项

Username ?

登录服务器的用户 一般是 root ③

① 必填项

Remote Directory ?

链接到服务器的那个目录下 ④

☐ Avoid sending files that have not changed ?

高级 ^ ⑤ Edited

⑥ ☒ Use password authentication, or use a different key ?

Passphrase / Password ?

服务器的登录密码 ⑦

Path to key ?

Key ?

Jump host ?

3. 如果没有使用jenkins默认的目录，而是像我一样单独用了一个目录。一开始写好shell直接构建会报错，因为原本的文件夹是空的，没有初始化git，在shell中直接编写 `git pull` 会出错。先 `git clone/git pull` 一下。

这不是唯一的方案，你可以写好dockerfile使用容器运行服务，也可以不通过容器部署jenkins，直接在宿主机部署jenkins。

3. 几个例子

下面是几个例子，其他项目也可以参考实现

3.1 GO项目

以GO项目为例，通过jenkins自动构建和部署go的后端

可能出错的地方：配置文件的位置

步骤：

1. 下载相关插件GO， 系统管理->全局工具配置 设置好go的环境。可以设置国内镜像，从官网下可能连接不上，下载好了，但是构建过程中显示找不到，就是环境变量没设置，需要到系统管理->系统配置 设置环境变量；也可以直接在shell中export，如下面的图片
2. 设置好gitee令牌、webhook等，参考上面的链接
3. 配置好go environment（在项目的配置中）

4. 编写shell 高级配置,需要配置ssh: 例如我这里已经做了挂载目录了, 所以容器内的文件变化会自动映射到容器外, 在容器内设置好GO的环境, 进行 go build , build好的exe文件已经在宿主机上了。再在ssh那里运行文件, 开启服务。

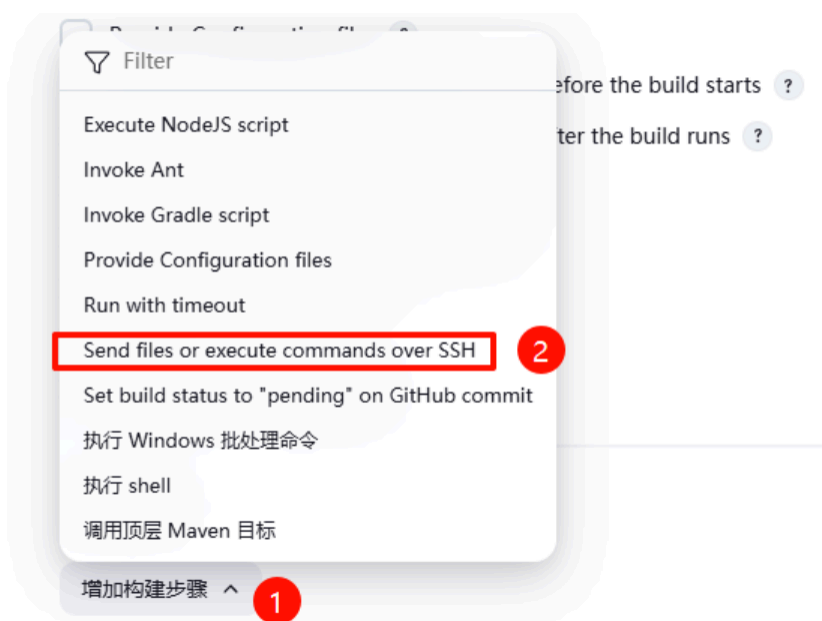
≡ 执行 shell ?

命令

[查看 可用的环境变量列表](#)

```
mkdir -p /service/uwebsite/backend
cd /service/uwebsite/backend
git pull origin master
chmod 777 /var/jenkins_home/tools/org.jenkinsci.plugins.golang.GolangInstallation/go/go
chmod 777 /service/uwebsite/backend
export GOROOT=/var/jenkins_home/tools/org.jenkinsci.plugins.golang.GolangInstallation/go/go
export PATH=$PATH:$GOROOT/bin
$GOROOT/bin/go version
export GOPROXY=https://goproxy.cn,direct
export GOSUMDB=sum.golang.google.cn
ls
go mod tidy
go build .
```

高级 ▾



Exec command ?

```
cd /service/uwebsite/backend
nohup ./UISwebsite_backend > /var/log/uwebsite.log 2>&1 &
```

用nohup保护进程, 如果服务没有正常启动, 可以查看日志。

3.2 前端项目

- 目前的情况: 在本地打包前端项目, 打包好上传到服务器, 用 nginx 进行转发。其他的静态资源也是直接上传服务器, 配置好nginx文件, 重启nginx

- 预期：nginx配置文件也写到gitee仓库，每次新增静态资源/修改前端代码，修改配置文件，push代码，根据这个配置文件，开启前端服务。

步骤：

1. 和上面类似的，装好nodejs和npm的环境



The image shows a configuration window for Node.js installation. At the top, there is a checked checkbox labeled 'Provide Node & npm bin/ folder to PATH'. Below this, the 'NodeJS Installation' section has a dropdown menu set to 'nodejs'. The 'npmrc file' section has a dropdown menu set to '- use system default -'. The 'Cache location' section has a dropdown menu set to 'Default (~/.npm or %APP_DATA%\npm-cache)'. At the bottom, there are two unchecked checkboxes: 'Set up Go programming language tools ?' and 'Terminate a build if it's stuck'.

2. 设置好gitee令牌、webhook等

3. 编写shell

打包前端项目，现在打包好的dist文件在根目录下

```
mkdir -p /service/uiswebsite/frontend
chmod 777 /service/uiswebsite/frontend

# 添加 Git 安全目录例外
git config --global --add safe.directory /service/uiswebsite/frontend

cd /service/uiswebsite/frontend
git pull origin master
npm install
npm run build
```

目前的nginx conf文件如下，我将其余的静态资源放到other， / 指向打包完成的dist目录

```

# 主应用（监听 80 端口）
server {
    listen 80;
    server_name 47.121.206.97 bjtuuis.cn www.bjtuuis.cn;
    client_max_body_size 100M;

    location / {
        root /service/uiswebsite/frontend/dist; # 主应用静态资源目录
        try_files $uri $uri/ /index.html;
    }

    location /api {
        proxy_pass http://localhost:8000; # 后端 API 代理
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}

# 倒计时应用（监听 82 端口）
server {
    listen 82;
    server_name 47.121.206.97 bjtuuis.cn www.bjtuuis.cn;
    client_max_body_size 100M;

    location / {
        root /service/uiswebsite/frontend/other/swcountdown; # 倒计时应用静态资源目录
        try_files $uri $uri/ /countdown.html;
    }
}

# 第三个应用（监听 81 端口）
server {
    listen 81;
    server_name 47.121.206.97 bjtuuis.cn www.bjtuuis.cn;
    client_max_body_size 100M;

    location / {
        root /service/uiswebsite/frontend/other/05f2917ab5c92d464ddd81ecc14f7ccf_28fa9713607
        try_files $uri $uri/ /index.html;
    }
}

```

编写ssh执行命令

Exec command ?

```

mv -f /service/uiswebsite/frontend/front.conf /etc/nginx/sites-available/front.conf
sudo ln -s /etc/nginx/sites-available/front.conf /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx

```